192425037

EXP25:

```
#include <stdio.h>

typedef unsigned long long u64;

long long gcd(long long a, long long b) { return b == 0 ? a : gcd(b, a % b); }

long long egcd(long long a, long long b, long long *x, long long *y) {
    long long x1, y1, g;
    if (b == 0) { *x = 1; *y = 0; return a; }
    g = egcd(b, a % b, &x1, &y1);
    *x = y1; *y = x1 - (a / b) * y1;
    return g;
}

u64 modpow(u64 b, u64 e, u64 m) {
    u64 r = 1; b %= m;
    while (e) { if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; }
    return r;
}

int main() {
    long long n, e, c[20], p = 0, q, phi, x, y, d;
    int cnt, i;

    printf("Enter n and e: ");
    scanf("%lld %lld", &n, &e);
    printf("Number of ciphertext blocks: ");
    scanf("%d", &cnt);
```

```
    printf("Enter the ciphertext blocks: ");
    for (i = 0; i < cnt; i++) scanf("%lld", &c[i]);

    /* If plaintext block M shares a factor with n, then C = M^e mod n shares it too,
       so gcd(C, n) reveals p or q and n can be factored. */
    for (i = 0; i < cnt; i++) {
        long long g = gcd(c[i], n);
        printf("gcd(C%d = %lld, n) = %lld\n", i + 1, c[i], g);
        if (g > 1 && g < n) { p = g; break; }
    }
    if (!p) { printf("No common factor found - this does not help.\n"); return 0; }

    q = n / p; phi = (p - 1) * (q - 1);
    printf("\nn factored: p = %lld, q = %lld, phi(n) = %lld\n", p, q, phi);
    egcd(e, phi, &x, &y);
    d = ((x % phi) + phi) % phi;
    printf("Private key d = %lld\n", d);
    printf("Recovered plaintext blocks: ");
    for (i = 0; i < cnt; i++) printf("%llu ", modpow(c[i], d, n));
    printf("\nYes - it helps: the whole message can be decrypted.\n");
    return 0;
}
```

OUTPUT:

```
Enter n and e: 3599 31
Number of ciphertext blocks: 3
Enter the ciphertext blocks: 624 3422 2195
gcd(C1 = 624, n) = 1
gcd(C2 = 3422, n) = 59

n factored: p = 59, q = 61, phi(n) = 3480
Private key d = 3031
Recovered plaintext blocks: 1234 177 2500
Yes - it helps: the whole message can be decrypted.
```